

Labo 05 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — 950 Mo refusés par la sécurité

Réponse. (1) **Le code source est dans l'image** : quiconque tire l'image lit l'application — et souvent des fichiers de configuration locaux. (2) **La surface d'attaque** : JDK, Maven, `git`, `curl`, un shell complet, des centaines de paquets Debian — autant d'outils pour un attaquant qui obtient l'exécution de code, et un rapport de scan à 300 CVE que personne ne traitera. (3) **Le dépôt** `~/m2` contient les artefacts téléchargés et, fréquemment, un `settings.xml` avec les identifiants du dépôt Maven privé. Le plus difficile à corriger sans multi-stage est (2) : on peut `rm` les sources et `.m2` (mal : les couches les gardent, labo 02), mais on ne peut pas retirer le JDK d'une image qui part d'une image JDK.

Pourquoi. L'image finale est le dernier `FROM` plus ce qu'on y ajoute ; on ne peut pas « soustraire » l'image de base. Seul un second `FROM` sur une base minimale, avec `COPY --from`, change de base.

Nuance. La taille elle-même a un coût opérationnel (temps de pull sur incident, stockage du registry), mais c'est l'argument le plus faible face à un responsable sécurité — le contenu compte plus que le poids.

Exemple.

```
podman run --rm --entrypoint sh api-mono:1.0 -c 'ls /app; javac -version; ls ~/m2 2>/dev/null'
```

Question 2 — Trois `FROM`

Réponse. Le **dernier** `FROM` produit l'image finale ; les deux autres sont des environnements temporaires, détruits à la fin du build (seul leur cache subsiste). Si aucun `COPY --from` (ni `FROM` ultérieur) ne référence le deuxième stage, Buildah **ne le construit pas du tout** : il l'ignore. On le voit dans la sortie : les stages sont numérotés `[1/3]`, `[2/3]`, `[3/3]`, et le numéro du stage inutile n'apparaît jamais.

Pourquoi. Le moteur calcule d'abord le graphe des dépendances entre stages à partir des `--from`, puis ne construit que ce qui mène à la cible (le dernier stage, ou `--target`).

Nuance. BuildKit fait la même chose et construit en plus les stages indépendants en parallèle. Un stage « inutile » a un usage légitime : un stage de tests (`RUN mvn test`) qu'on ne construit qu'avec `--target test` en CI.

Exemple.

```
podman build -f Dockerfile.unused -t u . 2>&1 | grep STEP      # [2/3] et [3/3] seulement,
jamais [1/3]
```

Question 3 — 420 Mo et les sources

Réponse. `COPY --from=build /app /app` recopie **tout le répertoire de travail** du stage `build` : `Api.java` / `src`, `pom.xml`, `target/` avec ses classes, et le JAR. Correction : ne copier que

l'artefact.

```
COPY --from=build /app/target/api.jar /app/api.jar
```

(et adapter l' `ENTRYPOINT` à `/app/api.jar`.)

Pourquoi. Le multi-stage ne filtre rien par lui-même : il ne met dans l'image finale que ce que `COPY --from` demande. Demander un dossier, c'est demander son contenu entier.

Nuance. Même corrigée, l'image contient encore un `.m2` ? Non — `.m2` est dans `/root` du stage build, pas dans `/app`. Mais c'est une chance, pas une garantie : la règle est de copier des **fichiers nommés**.

Exemple.

```
podman run --rm --entrypoint ls api-multi:1.0 /src      # No such file or directory : bon signe
```

Question 4 — `ng serve` en production

Réponse. Quatre raisons : (1) `ng serve` est un serveur de **développement** — non optimisé, sans compression ni cache HTTP, explicitement documenté comme non prévu pour la production ; (2) l'image contient **Node, les sources et `node_modules`** (souvent > 1 Go) : surface d'attaque et fuite de code ; (3) le build n'est pas fait **une fois** mais à chaque démarrage, en mode « watch », avec les *source maps* activées ; (4) aucune séparation entre l'application et son outillage — une dépendance de développement vulnérable est en production. À la place : `ng build --configuration production` dans un stage Node, puis `COPY --from` du dossier `dist/.../browser` dans une image `nginx:alpine` avec une configuration nginx qui renvoie `index.html` pour les routes Angular.

Pourquoi. Un front Angular compilé est statique. Le servir ne demande qu'un serveur de fichiers ; tout le reste est du build, qui appartient au stage jeté.

Nuance. Il existe un cas où Node reste en production : le rendu côté serveur (Angular SSR / Universal). C'est alors une **autre** application, avec son propre Dockerfile, et toujours pas `ng serve`.

Exemple.

```
podman images --format '{{.Repository}} {{.Size}}' | grep -E 'web-multi|node'      # 64 Mo contre 167 Mo (sans node_modules !)
```

Question 5 — `UnsatisfiedLinkError` après Alpine

Réponse. La génération de PDF s'appuie très probablement sur une **bibliothèque native** (`.so`) compilée pour `glibc` — police, rendu, compression. Alpine fournit `musl` : le chargeur refuse la bibliothèque, Java lève `UnsatisfiedLinkError`. La commande qui l'aurait montré : `ldd --version` dans chaque image (`musl libc` contre `GLIBC`). La migration aurait dû être testée avec les **traitements réels** (pas seulement `/actuator/health`), sur un environnement de recette, avec un plan de retour — et la décision prise composant par composant.

Pourquoi. Une bibliothèque native est liée à une `libc` précise ; ce n'est pas du bytecode portable. Deux semaines de délai, parce que la génération de PDF ne tourne peut-être qu'en fin de mois.

Nuance. Des alternatives existent : l'image `eclipse-temurin:21-jre-ubi9-minimal` (Red Hat, `glibc`, ~100 Mo), ou installer `gcompat` sur Alpine (fragile). Et ce n'est pas propre à Java : Python, Node avec des modules natifs, ont exactement le même piège.

Exemple.

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'ldd --version 2>&1 | head -1' # musl libc
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'ldd --version | head -1' # GLIBC 2.xx
```

Question 6 — Multi-stage et secrets

Réponse. Un `rm` crée une couche qui masque le fichier ; la couche qui l'a écrit reste dans l'image (labo 02). Un stage jeté, lui, n'est **pas** dans l'image finale : aucune de ses couches n'y figure. Si le secret n'a été écrit que dans un stage jeté, il n'existe nulle part dans l'artefact publié. Le multi-stage ne protège pas si le secret est **copié** dans le stage final (`COPY --from=build /app` avec le secret dedans), ou si l'artefact lui-même l'a absorbé (un `application.yml` avec mot de passe empaqueté dans le JAR), ou si le secret transite par un `ARG` du stage final (visible dans `history`).

Pourquoi. L'image finale = couches du dernier `FROM` + couches produites par ses instructions. Un stage précédent n'y contribue que par ce qu'un `COPY --from` en extrait.

Nuance. La solution moderne est `RUN --mount=type=secret` : le secret est disponible pendant une seule instruction, dans n'importe quel stage, sans jamais devenir une couche. Le multi-stage reste la garantie structurelle, le *secret mount* la garantie instruction par instruction.

Exemple.

```
podman build --secret id=pw,src=pw.txt -f Dockerfile.secret -t sec .
podman run --rm sec ls /run/secrets # No such file or directory
```

Question 7 — JAR en bloc contre couches

Réponse. (a) Une couche de 50 Mo qui change à chaque build : le `push` et chaque `pull` transfèrent **50 Mo**. (b) Quatre couches : dépendances (~45 Mo, inchangées), loader (~1 Mo, inchangé), snapshots (0), application (~5 Mo) : le déploiement transfère **~5 Mo**. Ratio 10. (a) reste acceptable parce que 50 Mo sur un réseau de datacenter prennent une seconde, que le JRE (180 Mo) est de toute façon partagé, et que (b) ajoute un stage, un `ENTRYPOINT` différent (`org.springframework.boot.loader...` ou `java -jar` sur le dossier) et de la complexité à expliquer.

Pourquoi. Le transfert est différentiel par couche ; ce qui compte est la taille de la couche qui change, pas celle de l'image.

Nuance. (b) devient rentable quand on déploie souvent sur beaucoup de nœuds, ou sur un réseau lent (edge, sites distants). Et le principe s'applique sans Spring : séparer `lib/` (stable) et `classes/` (volatil) suffit.

Exemple.

```
podman history api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -6 # une couche de 50 Mo, ou quatre couches
```

Question 8 — 90 secondes devenues 7 minutes

Réponse. Le nouvel agent a un **cache vide** : le cache de build (couches) vit sur la machine qui construit, et un agent neuf — ou un agent éphémère recréé à chaque pipeline — repart de zéro. Les 5 minutes de `dependency:go-offline` sont donc repayées. Deux mécanismes : (1) un **cache mount** (`RUN --mount=type=cache,target=/root/.m2`), qui conserve le dépôt Maven sur l'agent entre les builds, même quand `pom.xml` change ; (2) un **cache distant** — `--cache-from / --cache-to` vers le registry — qui permet à un agent neuf de récupérer les couches d'un build précédent. Avec Podman rootless, le cache (couches et *cache mounts*) est dans `~/.local/share/containers/storage` de l'utilisateur qui construit : un agent qui exécute chaque job sous un utilisateur ou un `home` différent, ou qui détruit son `home`, n'a jamais de cache.

Pourquoi. « Rien n'a changé » est vrai côté sources, faux côté cache : le cache est un état local de la machine, pas une propriété du Dockerfile.

Nuance. Les agents éphémères sont voulus (isolation, reproductibilité) ; la réponse est de rendre le cache **explicite et externe**, pas de garder des agents longue durée. Et un `--no-cache` dans le pipeline « pour être sûr » produit exactement ce symptôme, en permanence.

Exemple.

```
podman build --cache-to registry.interne/monapp/api-cache --cache-from
registry.interne/monapp/api-cache -t api:1.5 .
podman info --format '{{.Store.GraphRoot}}' # où vit le cache de cet utilisateur
```

Question 9 — Ce qu'on perd avec distroless

Réponse. (1) `podman exec -it ... sh` : pas de shell, donc pas d'exploration interactive, pas de `cat` d'un fichier de configuration, pas de `curl localhost:8080/actuator`. Compensation : des endpoints d'observabilité exposés (`/actuator/health`, `/info`, `/env`), des logs structurés et complets sur `stdout`, et `podman cp` pour extraire un fichier. (2) **Les outils de diagnostic** (`jcmd`, `jstack`, `ps`, `netstat`) : rien pour prendre un *thread dump* ou voir les sockets. Compensation : un conteneur *sidecar* de debug qui partage les namespaces (`podman run --pid=container:api --network=container:api debug-image`), ou les outils JMX/Actuator (`/actuator/threaddump`) exposés au réseau interne.

Pourquoi. Tout ce que l'exploitant utilise pour entrer dans un conteneur, un attaquant l'utilise aussi. Distroless supprime les deux à la fois ; il faut donc déplacer l'observabilité **hors** de l'image.

Nuance. Les images distroless existent en variante `:debug` avec un shell busybox — utile en recette, interdite en production. Et Kubernetes propose `kubectrl debug` avec conteneurs éphémères pour exactement ce besoin.

Exemple.

```
podman exec d sh -c ls # executable file `sh` not found
curl -s localhost:18082/actuator/health # l'observabilité passe par HTTP
```

Question 10 — Le cache mount, VOLUME, et # syntax=

Réponse. Les données sont dans un **dossier de cache géré par le moteur de build** (BuildKit ou Buildah), sur la machine qui construit — chez Podman rootless, dans votre stockage utilisateur. Il est monté dans le conteneur de build **pendant l'instruction seulement**, puis démonté : rien

n'est écrit dans une couche, donc rien dans l'image. Un `VOLUME` est l'inverse : une déclaration dans l'image qui, à l'**exécution**, crée un volume pour le conteneur ; il n'a aucun effet pendant le build. Sans `# syntax=docker/dockerfile:1` : sous Docker (versions récentes, BuildKit par défaut), `--mount` fonctionne de toute façon avec la syntaxe stable actuelle ; la ligne ne servait qu'à forcer une version plus récente du frontend. Sous Podman, la ligne est **ignorée** (Buildah n'a pas de frontend) et `--mount` fonctionne nativement.

Pourquoi. Le cache mount est un mécanisme du moteur de build ; le `VOLUME` un mécanisme du runtime. Ils ne partagent que le mot « montage ».

Nuance. Le cache mount n'est pas partagé entre machines ni entre utilisateurs, et il n'est pas invalidé par le contenu : un dépôt Maven corrompu y reste. `podman system prune --build-cache ...` n'existe pas encore : on supprime le stockage ou on utilise `id=` pour changer de cache.

Exemple.

```
podman build --no-cache -f Dockerfile.cache -t c . 2>&1 | grep dep-      # les marqueurs
s'accumulent d'un build à l'autre
podman run --rm c ls /root/.m2                                         # absent de l'image
```

Question 11 — « Le multi-stage ne sert à rien pour Angular »

Réponse. Sans multi-stage, l'image finale est l'image dans laquelle `ng build` a tourné : `node:22-alpine` (~170 Mo) **plus** `node_modules` (500 Mo à 1 Go) **plus** les sources TypeScript **plus** le `dist/` — et il faut encore y ajouter un serveur pour servir `dist/`. Avec multi-stage : `nginx:alpine` (64 Mo) plus quelques Mo de fichiers statiques. L'écart est de 10 à 20 fois, et le contenu change de nature : plus de Node, plus de sources, plus de dépendances de build.

Pourquoi. Le fait que le *résultat* soit statique est précisément l'argument **pour** le multi-stage : puisque l'exécution n'a besoin de rien de ce qui a servi à construire, autant ne rien garder.

Nuance. Sans conteneur, une équipe peut aussi faire `ng build` en CI et copier `dist/` dans une image nginx en une seule étape (`COPY dist/ /usr/share/nginx/html`). C'est un « multi-stage » dont le premier stage est la CI — valable, mais le build n'est plus reproductible depuis le seul Dockerfile.

Exemple.

```
podman images --format '{{.Repository}} {{.Size}}' | grep -E 'web-multi|node'      # 64.2 MB
contre 167 MB
```

Question 12 — `/app/dist: no such file or directory`

Réponse. Le stage `build` a `WORKDIR /src` : le build produit `/src/dist`, pas `/app/dist`.
Correction : `COPY --from=build /src/dist/<projet>/browser /usr/share/nginx/html` (le sous-dossier dépend de la version d'Angular et du nom du projet). Pour diagnostiquer sans deviner :
`podman build --target build -t dbg .` puis `podman run --rm dbg find / -name index.html -path '*dist*'.`

Pourquoi. `COPY --from` copie depuis le système de fichiers du stage, avec des chemins **absolus** de ce stage. Une erreur de `WORKDIR` ou de structure `dist/` est invisible tant qu'on ne regarde pas dedans.

Nuance. Depuis Angular 17, la sortie par défaut est `dist/<projet>/browser/` ; avant, `dist/<projet>/`. Le `--target` évite de dépendre de sa mémoire.

Exemple.

```
podman build --target build -t dbg . && podman run --rm dbg ls -R /src/dist | head
```

Question 13 — `RUN mvn test` dans le Dockerfile

Réponse. Dans le stage de build, **après** la compilation et **avant** le `package` (ou en un seul `mvn package` sans `-DskipTests`) : si un test échoue, `RUN` échoue, le build s'arrête, aucune image n'est produite. Inconvénient : les tests tournent dans un conteneur de build isolé — pas de rapport JUnit exploitable par la CI (il est dans un stage jeté, sauf à le copier avec `--target` ou `--output`), pas de base de test facilement joignable (Testcontainers a besoin d'un moteur), et le temps de build de l'image inclut celui des tests, même quand on ne voulait que reconstruire.

Pourquoi. Le Dockerfile est un bon garant (« aucune image sans tests verts ») mais un mauvais outil de reporting.

Nuance. Le compromis courant : la CI lance les tests **et** le build d'image en deux jobs, avec le build conditionné au succès des tests ; le Dockerfile garde `-DskipTests` pour rester rapide. On obtient le rapport et la garantie, au prix d'une dépendance à la CI.

Exemple.

```
RUN mvn -q test                # rouge -> le build s'arrête ici
RUN mvn -q package -DskipTests
```

Question 14 — Une couche de 250 Mo ou cinq couches de 280 Mo

Réponse. L'image de **280 Mo en cinq couches** se déploie plus vite pour une mise à jour du code : seule la couche volatile (quelques Mo) est transférée, les quatre autres sont déjà sur les nœuds et dans le registry. L'image de 250 Mo en une couche retransfère 250 Mo à chaque version. La réponse s'inverse quand les nœuds n'ont **rien** (premier déploiement, nœud neuf, registry vidé, ou stratégie de tags qui change tout à chaque fois) : là, $250 < 280$, et la couche unique gagne — de peu.

Pourquoi. Le coût de transfert est celui des couches manquantes, pas de l'image. La stabilité des couches vaut plus que leur nombre.

Nuance. `--squash` (Buildah) ou une base minimale peuvent ramener les 280 Mo à 250 sans perdre les couches : les deux critères ne sont pas exclusifs. Et le gain n'existe que si les couches stables sont **identiques bit à bit** d'un build à l'autre — reproductibilité du build oblige (pas de `apt-get update` non épinglé dans une couche « stable »).

Exemple.

```
podman push registry.interne/monapp/api:1.5.1    # blobs stables : instantanés ; seule la
couche du code est copiée
```